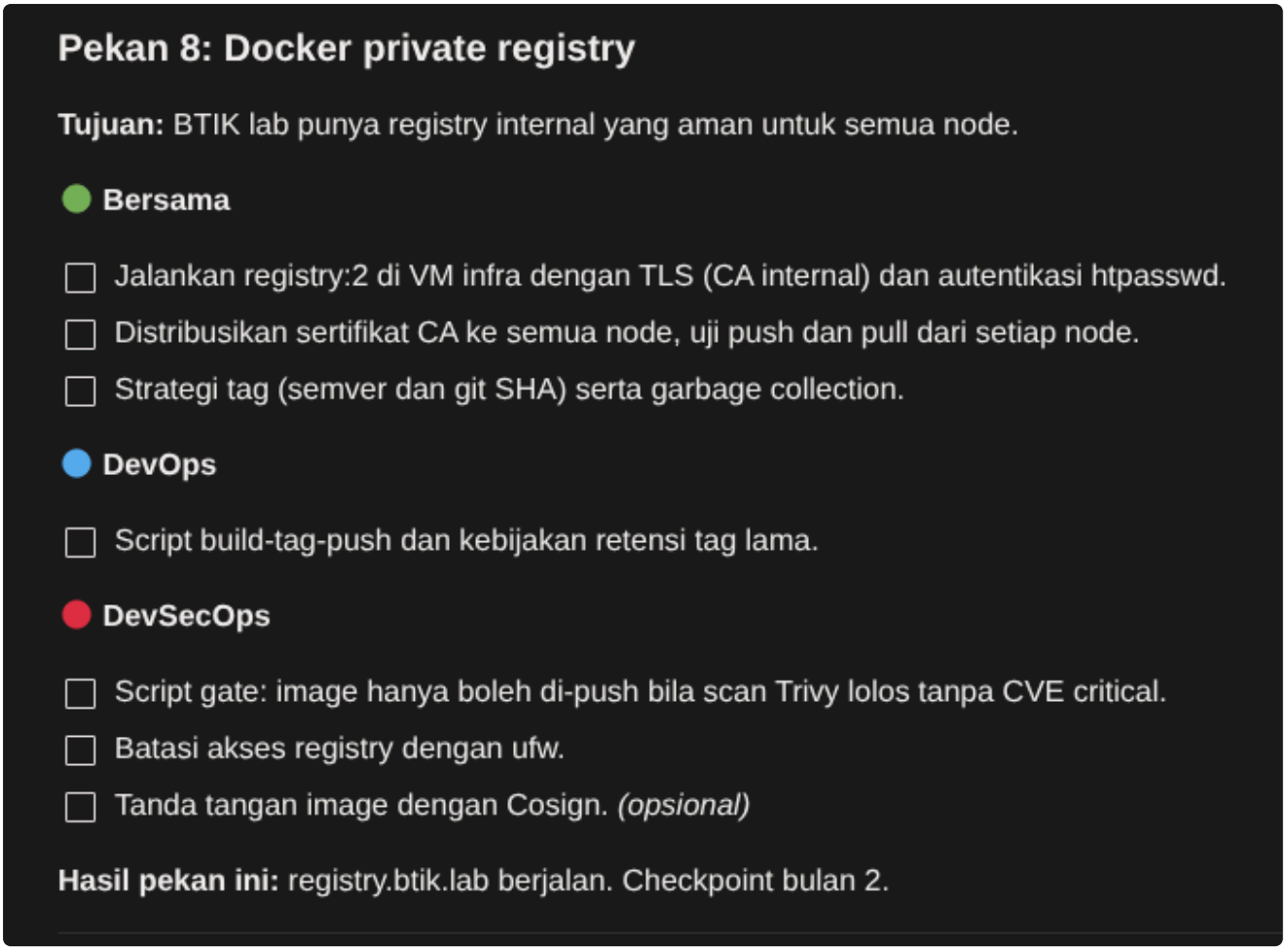
Dokumentasi Task 8



langsung aja kita mulai dari **private registry**, nah intinya itu alasan kenapa private registry digunakan adalah saat kita mendevelop berbagai image yang mana image image itu berkaitan dengan aplikasi tertentu yang mana itu hanya internal yang tau ya tidak mungkin kita mengunggah image docker buatan kita ke docker registry yang mana itu siapa saja bisa melakukan pull dll kan dan solusinya itu adalah kita menggunakan private registry yang mana intinya banget kita membuat sebuah registry dalam container yang ada di dalam infra kita sendiri dengan menggunakan docker agar image image tertentu bisa dipull dari sebuah registry tanpa harus mengunggah image tersebut pada registry umum, langsung aja kita akan ke vm infra untuk menjalankan registry:2 dan TLS yang mana registry:2 sendiri merupakan image yang disediakan oleh docker nya sendiri agar kita bisa membuat sebuah private registry sendiri lalu kenapa TLS juga? karena by default docker itu menolak untuk berkomunikasi dengan registry melalui http biasa dan yang seperti itu tuh disebut dengan insecure registry yang mana maka dari itu langsung aja kita akan ke vm infra dan kita akan generate TLS dulu dengan command

```
sudo apt install libnss3-tools -y
curl -JLO "https://dl.filippo.io/mkcert/latest?for=linux/amd64"
chmod +x mkcert-v*-linux-amd64
```

```
sudo mv mkcert-v*-linux-amd64 /usr/local/bin/mkcert
mkcert -install
```

lalu

```
mkdir -p ~/registry/certs
cd ~/registry/certs
mkcert -cert-file registry.crt -key-file registry.key registry.btik.lab
localhost 127.0.0.1 $(hostname -I | awk '{print $1}')
```

lalu kita akan coba cari lokasi ca root nya dengan command

```
mkcert -CAROOT
```

lalu jika semua command itu dijalankan maka akan menghasilkan output



lalu hasil dari mkcert -CAROOT adalah



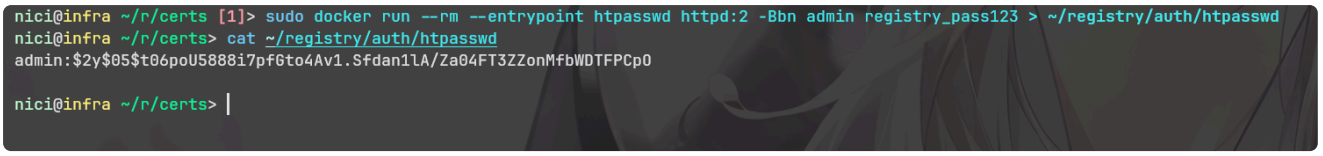
nah selanjutnya kita akan generate kredensial pada htpassword untuk registry nya dan kita akan membuat sebuah direktori dengan command

```
mkdir -p ~/registry/auth
```

nah lalu kita akan generate file htpasswd dengan menggunakan docker dengan menggunakan image http yang mana pada image tersebut udah punya tools htpasswd bawaan dengan command

```
docker run --rm --entrypoint htpasswd httpd:2 -Bbn admin registry_pass123
> ~/registry/auth/htpasswd
```

lalu saat kita verifikasi maka hasilnya kira kira seperti ini



nah disitu ada hash yang udah tercipta lalu kita akan langsung menjalankan container registry dengan command

```
docker run -d \
  --name registry \
  --restart unless-stopped \
  --network bridge \
  -p 443:443 \
  -e REGISTRY_HTTP_ADDR='0.0.0.0:443' \
  -e REGISTRY_HTTP_TLS_CERTIFICATE='/certs/registry.crt' \
  -e REGISTRY_HTTP_TLS_KEY='/certs/registry.key' \
  -e REGISTRY_AUTH='htpasswd' \
  -e REGISTRY_AUTH_HTPASSWD_REALM='Registry Realm' \
  -e REGISTRY_AUTH_HTPASSWD_PATH='/auth/htpasswd' \
  -e REGISTRY_STORAGE_DELETE_ENABLED='true' \
  -v /home/nici/registry/auth:/auth \
  -v /home/nici/registry/certs:/certs \
  -v registry-data:/var/lib/registry \
  registry:2
```

lalu kita akan verifikasi dengan menggunakan command

```
docker ps | grep registry
docker logs registry
```

yang mana hasilnya kira kira seperti ini

```
nici@infra ~/r/certs [1]> sudo docker ps | grep registry
                          sudo docker logs registry
9b4c359021fa   registry:2   "/entrypoint.sh /etc…"   24 seconds ago   Up 24 seconds   0.0.0.0:443→443/tcp, [::]:443→443/tcp, 5000/tcp   registry
time="2026-10-15T07:25:06.677666519Z" level=warning msg="No HTTP secret provided - generated random secret. This may cause problems with uploads if multiple registrie
s are behind a load-balancer. To provide a shared secret, fill in http.secret in the configuration file or set the REGISTRY_HTTP_SECRET environment variable." go.vers
ion=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c service=registry version=2.8.3
time="2026-10-15T07:25:06.677788691Z" level=info msg="redis not configured" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c service=registry vers
ion=2.8.3
time="2026-10-15T07:25:06.677894574Z" level=info msg="using inmemory blob descriptor cache" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c servi
ce=registry version=2.8.3
time="2026-10-15T07:25:06.680386153Z" level=info msg="restricting TLS version to tls1.2 or higher" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9
c service=registry version=2.8.3
time="2026-10-15T07:25:06.681666684Z" level=info msg="restricting TLS cipher suites to: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,
TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA25
6,TLS_AES_128_GCM_SHA256,TLS_CHACHA20_POLY1305_SHA256,TLS_AES_256_GCM_SHA384" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c service=registry ve
rsion=2.8.3
time="2026-10-15T07:25:06.683500926Z" level=info msg="listening on [::]:443, tls" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c service=registr
y version=2.8.3
time="2026-10-15T07:25:06.681549985Z" level=info msg="Starting upload purge in 57m0s" go.version=go1.20.8 instance.id=e9ef51f6-7ff2-4aa7-b488-4cf23ca22e9c service=reg
istry version=2.8.3
nici@infra ~/r/certs>
```

nah kita akan langsung aja tes ombak dari lokal dulu dengan command

```
curl --cacert ~/registry/certs/registry.crt -u admin:registry_pass123
https://registry.btik.lab/v2/
```

yang jika berhasil akan menghasilkan {} yang mana hasil akhirnya adalah

```
nici@infra ~/r/certs [1]> curl --cacert ~/registry/certs/registry.crt -u admin:registry_pass123 https://registry.btik.lab/v2/
{}
nici@infra ~/r/certs>
```

nah disitu menghasilkan {} yang mana menandakan bahwa registry udah jalan aman tentram sentosa lalu kita akan coba ambil CAROOT dari infra, dan kita akan coba dari worker 2 deh dengan command(ini masih dijalankan pada vm infra)

```
mkcert -CAROOT
```

lalu kita akan catat path nya beserta isiannya dan hasilnya kira kira kaya gini

```
nici@infra ~/r/certs> ls /home/nici/.local/share/mkcert/
rootCA-key.pem  rootCA.pem
nici@infra ~/r/certs>
```

nah ini disitu ada 2 jenis file, rootCA-key.pem dan rootCA.pem dan private key kita itu yang rootCA-key.pem jadi gaboleh diapa apain ya jadi yang akan kita bagi ke orang host lain itu adalah public key nya yang mana itu adalah rootCA.pem

dan langsung aja kita copy rootCA ke vm yang lain dengan command

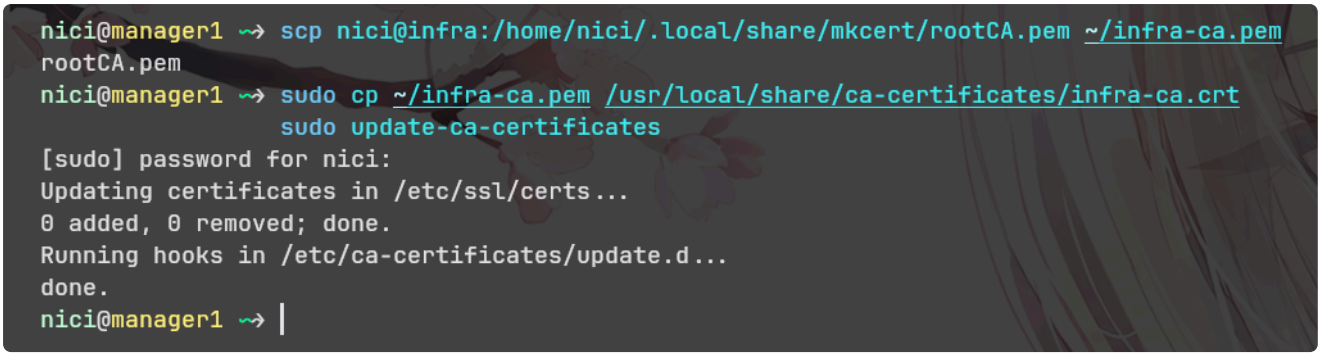
```
scp nici@infra:/home/nici/.local/share/mkcert/rootCA.pem ~/infra-ca.pem
```

nah setelah itu kita gunakan command

```
sudo cp ~/infra-ca.pem /usr/local/share/ca-certificates/infra-ca.crt
```

```
sudo update-ca-certificates
```

untuk memasukkan rootCA yang telah ter copy ke dalam sistem kita yang mana kalau berhasil akan menghasilkan output



lalu kita juga harus mendaftarkan domain registry nya ke setiap node dengan command

```
echo "192.168.56.13 registry.btik.lab" | sudo tee -a /etc/hosts
```

yang kira kira hasilnya seperti ini



setelah itu kita restart saja docker nya biar baca ulang trust store yang baru saja kita tambahkan dengan comman

```
sudo systemctl restart docker
```

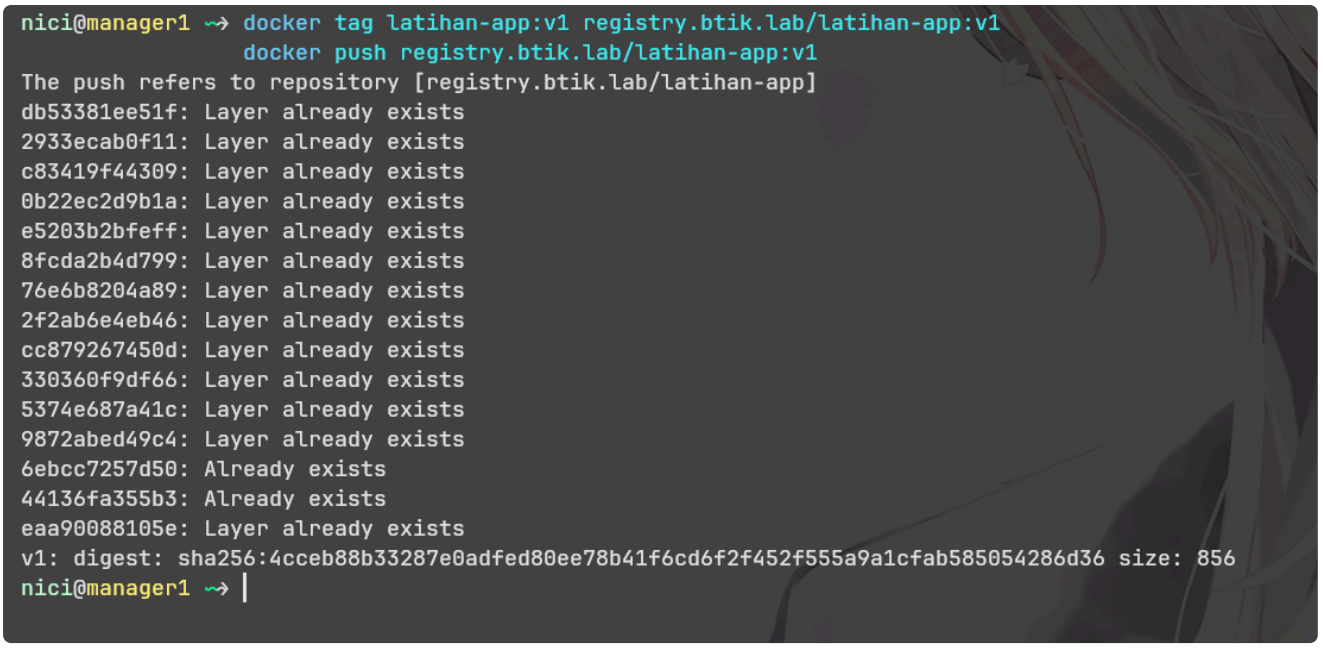
nah setelah itu kita langsung aja tes ombak dengan login dulu dengan command

```
docker login registry.btik.lab -u admin -p registry_pass123
```

lalu push dengan command

```
docker tag latihan-app:v1 registry.btik.lab/latihan-app:v1
docker push registry.btik.lab/latihan-app:v1
```

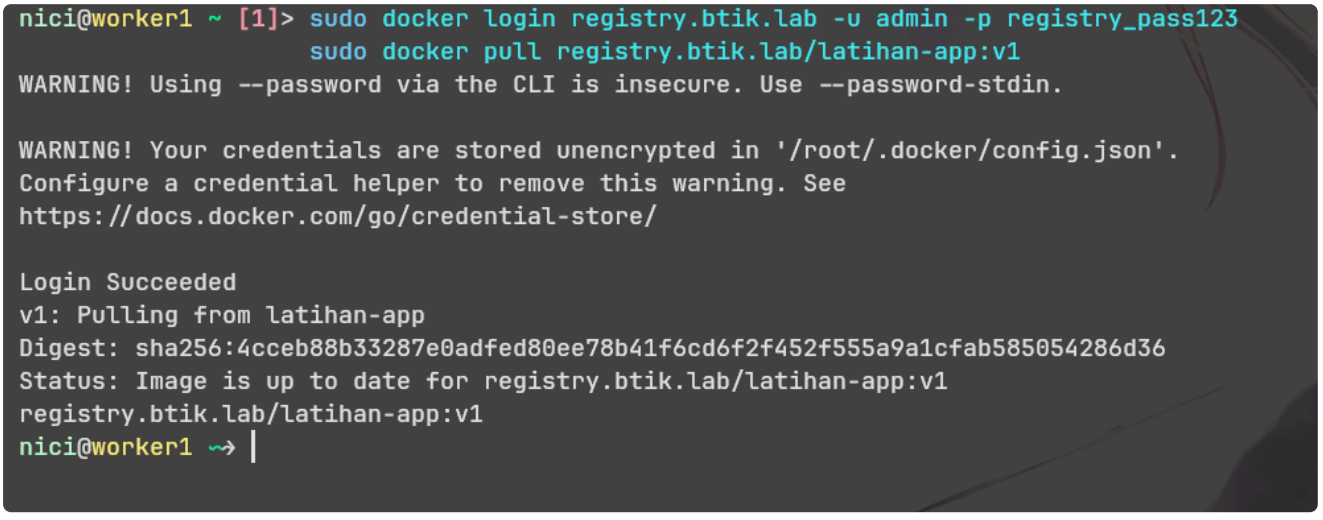
yang kira kira outputnya adalah



lalu uji pull dari vm worker 2 deh dengan command

```
docker login registry.btik.lab -u admin -p registry_pass123
docker pull registry.btik.lab/latihan-app:v1
```

nah akan menghasilkan output

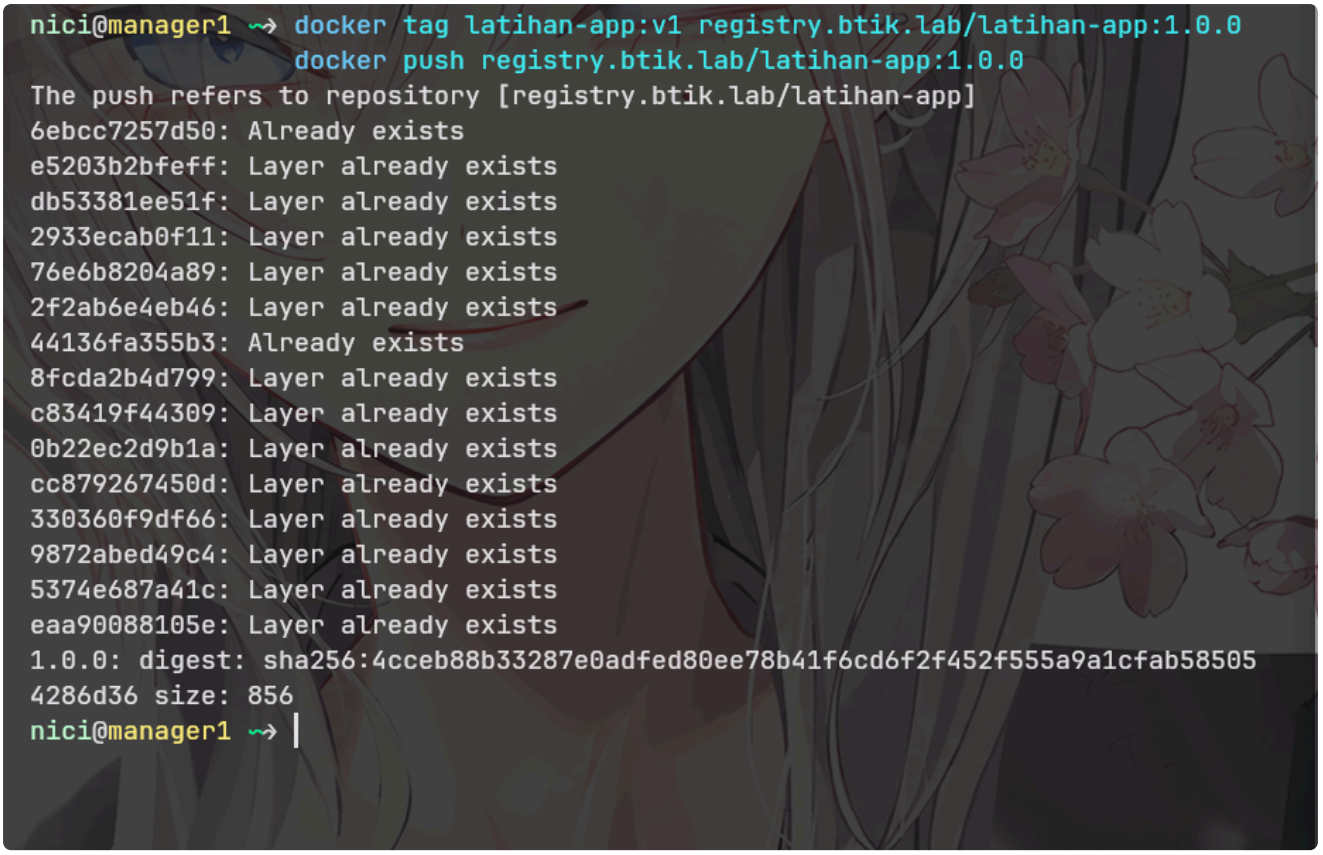


lalu kita akan masuk kek **strategi tag(semver sha256) serta garbage collection**

untuk strategi tag kurang lebih mirip dengan waktu task ke 4 yang mana itu ada beberapa model yang bisa aku gunakan saat membuat tag cuma ya bedanya ini kita ngasi tag ke sebuah registry cuma kurang lebih sama saja dan untuk strategi tag nya kurang lebih ada sematic version dan git sha, nah intinya banget kalo semver itu kan seperti yang ada pada task sebelum sebelumnya dan soal yang pake git sha itu juga untuk menandai image tersebut itu, kaya contoh ada pengembangan aplikasi a versi satu yang mana disitu kita membuat 2 tag pada image, yaitu semver yang mana itu terdiri dari nama image lalu :v1 dan yang git sha version itu kita ambil dari sha commit dari aplikasi tersebut, kira kira kaya gitu intinya dan untuk ngasi tag yang semver itu gampang, bisa pake command ini

```
docker tag latihan-app:v1 registry.btik.lab/latihan-app:1.0.0
docker push registry.btik.lab/latihan-app:1.0.0
```

yang kira kira hasilnya kaya gini



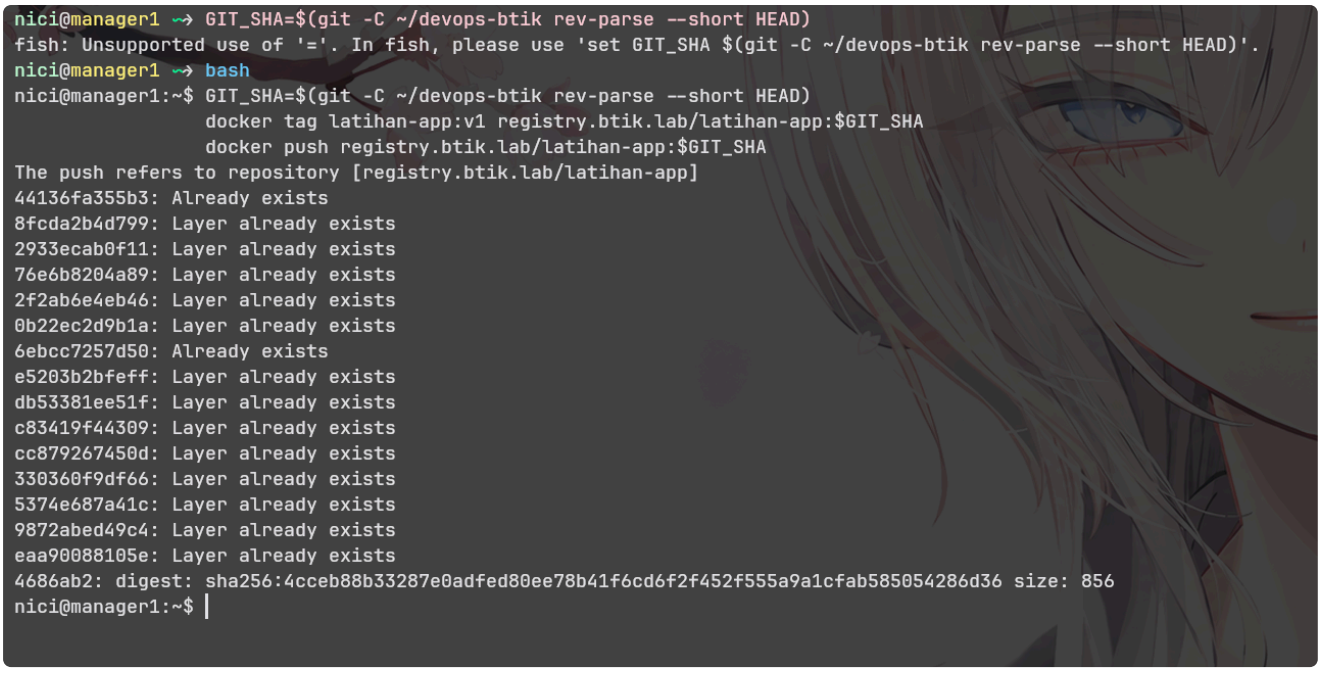
nah yang bagian git sha kita akan coba untuk git clone repo yang aku gunakan pada host, yaitu https://github.com/shooterman666/devops-btik/ ke vm yang nge push dan aku akan coba untuk ngasi tag sha berdasarkan commit dan karena direktori dimana image ada dan direktori repo nya berbeda jadi kita lakukan sedikit penyesuaian yang mana perintah yang akan digunakan adalah

```
GIT_SHA=$(git -C ~/devops-btik rev-parse --short HEAD)
```

```
docker tag latihan-app:v1 registry.btik.lab/latihan-app:$GIT_SHA
```

```
docker push registry.btik.lab/latihan-app:$GIT_SHA
```

yang mana hasilnya akan seperti ini kira kira



nah lanjut ke bagian garbage collection, hhmmm ini intinya banget itu adalah kita ngehapus layer fisik yang udah tidak digunakan oleh tag lain, contoh nih kita punya image yang ada dua tag image yang mana itu v1 dan v2, nah image sendiri kan terdiri dari beberapa layer kan? kalau kita lihat dari contoh Dockerfile dibawah ini

Dockerfile 1

```
FROM node:20
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["node", "app.js"]
```

Dockerfile 2

```
FROM node:20
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
RUN echo "Versi 2" > version.txt
CMD ["node", "app.js"]
```

nah disitu kan kita bisa melihat kalau kedua Dockerfile memiliki referensi manifest yang sama kecuali Dockerfile yang ke 2 yang mana dia memiliki manifest yang mana tidak ada pada manidest Dockerfile 1, yaitu RUN echo "Versi 2" > version.txt yang mana jika Dockerfile 1 itu di build dan aku berikan tag 1 dan begitu pula dengan Dockerfile 2 yang mana aku beri tag 2 dan jika aku menghapus tag 1 maka seluruh manifest tag 1 sendiri akan dihapus akan tetapi garbage collection yang mana aku sebut gc saja akan mengecek blob

mana yang direferensikan ke tag 2 yang mana ditemukan bahwa seluruh layer pada tag 1 manifest nya direferensikan ke tag 2 yang mana tidak ada layer yang dihapus dari tag 1 dan jika kita lakukan sebaliknya, maka gc menemukan bahwa bagian echo itu manifestnya tidak direferensikan ke manifest manapun alhasil bagian echo itu akan dihapus, nah langsung aja kita akan praktek yang mana kita akan cek dulu dengan command

```
curl --cacert ~/infra-ca.pem \
  -u "$REG_AUTH" -fsS \
  https://registry.btik.lab/v2/latihan-app/tags/list | jq .
```

itu buat cek isian registry dan hasilnya

```
curl --cacert ~/infra-ca.pem \
  -u "$REG_AUTH" -fsS \
  https://registry.btik.lab/v2/latihan-app/tags/list | jq .
Password registry:
{
  "name": "latihan-app",
  "tags": [
    "v1",
    "1.0.0",
    "4686ab2"
  ]
}
```

nah disitu kan ada v1 dll dan disitu aku akan menambahkan satu tag baru dengan command

```
docker pull registry.btik.lab/latihan-app:1.0.0
GC_DIR=$(mktemp -d)
cat > "$GC_DIR/Dockerfile" <<'EOF'
FROM registry.btik.lab/latihan-app:1.0.0
RUN printf 'Extra layer for garbage collection lab\n' > /app/gc-marker.txt
EOF
docker build \
  -t registry.btik.lab/latihan-app:2.0.0 \
  "$GC_DIR"
docker push registry.btik.lab/latihan-app:2.0.0
```

nah disitu kita nge pull dulu latihan-app:1.0.0 dan pada Dockerfile kita menambahkan baris baru dan kita build ulang lalu push ke registry dengan tag :2.0.0 nah dan saat kita cek ulang maka hasilnya

```
nici@manager1:~/latihan-dockerfile$ curl --cacert ~/infra-ca.pem \
  -u "$REG_AUTH" -fsS \
  https://registry.btik.lab/v2/latihan-app/tags/list | jq .
{
  "name": "latihan-app",
  "tags": [
    "2.0.0",
    "v1",
    "1.0.0",
    "4686ab2"
  ]
}
```

disitu ada sebuah tag baru yang muncul dan kita disini akan menghapus ke 3 tag yang lain, yaitu v1, 1.0.0, dan 468ab2 dengan command

```
curl --cacert ~/infra-ca.pem -u "$REG_AUTH" -sS -o /dev/null \
  -w 'HTTP %{http_code}\n' \
  -X DELETE \
  "https://registry.btik.lab/v2/latihan-
app/manifests/sha256:4cceb88b33287e0adfed80ee78b41f6cd6f2f452f555a9a1cfab5
85054286d36"
```

kenapa kita menghapus 3 tag hanya dengan 1 command dan satu digest? karena kalau kita lihat digest ke 4 tag tersebut yang mana adalah

```
done
v1          sha256:4cceb88b33287e0adfed80ee78b41f6cd6f2f452f555a9a1cfab585054286d36
1.0.0       sha256:4cceb88b33287e0adfed80ee78b41f6cd6f2f452f555a9a1cfab585054286d36
4686ab2     sha256:4cceb88b33287e0adfed80ee78b41f6cd6f2f452f555a9a1cfab585054286d36
2.0.0       sha256:0b7312c431e4d1329d3640fe87ddfdf05a5e0ad01a3e58ad7c0981b529d68a33
```

dari v1 sampai 468ab2 itu punya digest yang sama jadi itu langsung dihapus saja digest nya dan 3 tag langsung ilang lalu kita akan verifikasi dengan command

```
curl --cacert ~/infra-ca.pem -u "$REG_AUTH" -sS \
  https://registry.btik.lab/v2/latihan-app/tags/list
```

dan hasilnya adalah

```
nici@manager1:~/latihan-dockerfile$ curl --cacert ~/infra-ca.pem \
  -u "$REG_AUTH" -fsS \
  https://registry.btik.lab/v2/latihan-app/tags/list | jq .
{
  "name": "latihan-app",
  "tags": [
    "2.0.0"
  ]
}
```

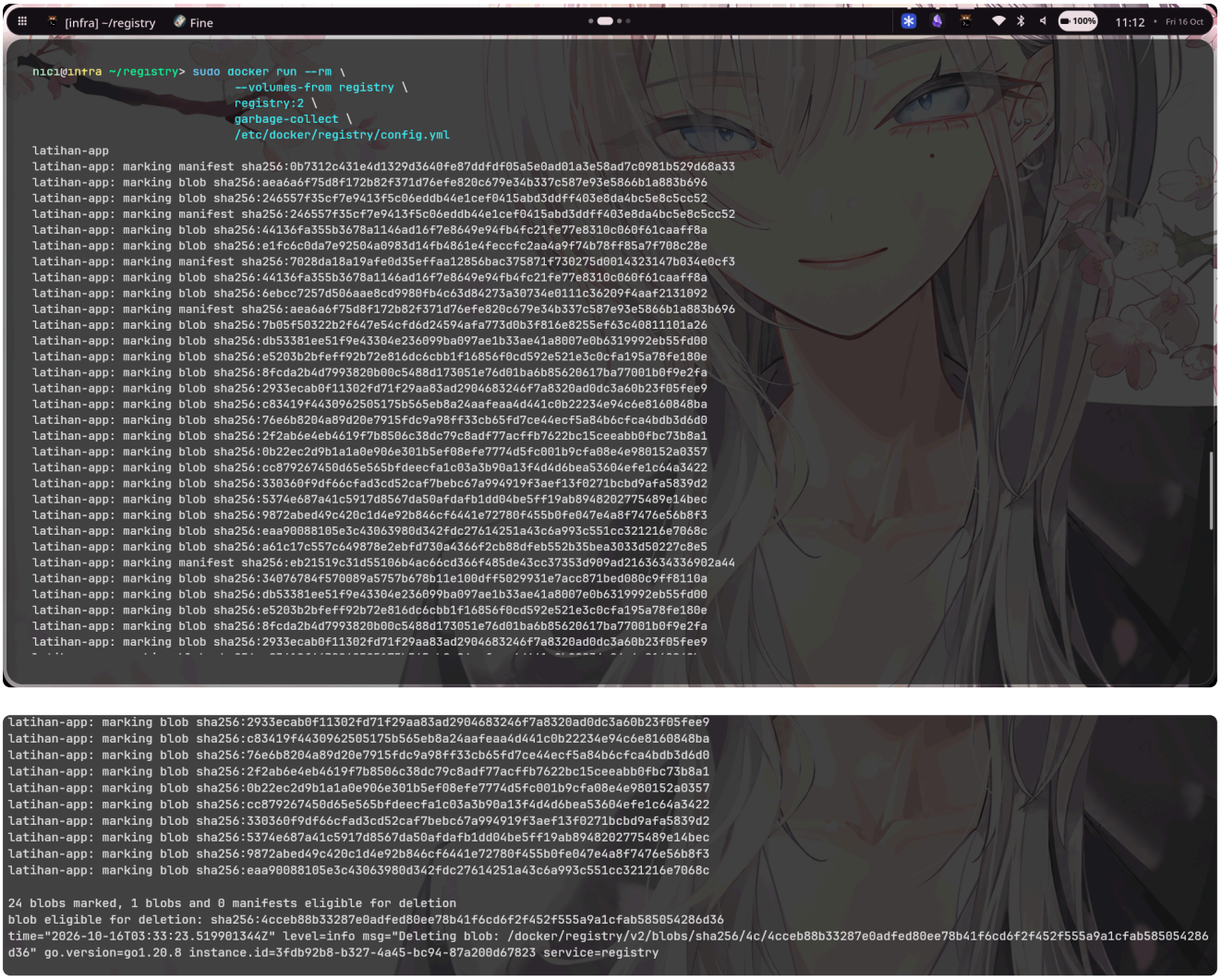
disitu tersisa satu saja yang masih ada dan kita juga akan menghentikan sementara registry karena kalau di environment yang nyata aku gunakan untuk jaga jaga saja agar saat proses gc masih berjalan, tidak ada manusia manusia yang ngepush jadi kita matikan dulu dengan command

```
sudo docker stop registry
```

dan lalu kita jalankan langsung gc ya dengan command

```
sudo docker run --rm \
  --volumes-from registry \
  registry:2 \
  garbage-collect \
  /etc/docker/registry/config.yml
```

yang mana outputnya kira kira kaya gini



karena semua layer yang digunakan pada dockerfile 1 itu manifest nya direferensikan ke dockerfile 2 jadinya hasilnya itu 0 manifest eligible for deletion lalu jangan lupa kita jalankan lagi registry nya dengan command

```
sudo docker start registry
```

oke cukup sudah untuk bagian garbage collection dan kita akan lanjut ke bagian **Script build-tag-push dan kebijakan retensi tag lama** yang mana kita intinya akan membuat skrip yang berfungsi untuk ngebuild, memberi tag dan nge push yang mana kira kira isi dari skripnya seperti ini

```
#!/bin/bash
set -euo pipefail

REGISTRY="registry.btik.lab"
IMAGE_NAME="latihan-app"

GIT_REPO_DIR="$HOME/devops-btik"
DOCKERFILE_DIR="$HOME/latihan-dockerfile"

# Versi bisa dikasih manual: ./build-push.sh 2.0.1
# Kalau nggak dikasih, ambil otomatis dari git (tag terakhir, atau short
commit hash kalau belum ada tag)
VERSION="${1:-}"

if [ -z "$VERSION" ]; then
  if [ ! -d "$GIT_REPO_DIR/.git" ]; then
    echo "Tidak ada argumen versi, dan $GIT_REPO_DIR bukan git repo."
    echo "Usage: $0 <semver-version>"
    exit 1
  fi

  VERSION="$(git -C "$GIT_REPO_DIR" describe --tags --always --dirty
2>/dev/null)"

  if [ -z "$VERSION" ]; then
    echo "Gagal mengambil versi dari git di $GIT_REPO_DIR"
    exit 1
  fi

  echo "Versi tidak diberikan, pakai versi dari git: ${VERSION}"
fi

if [ ! -f "$DOCKERFILE_DIR/Dockerfile" ]; then
  echo "Dockerfile tidak ditemukan di $DOCKERFILE_DIR"
  exit 1
fi

IMAGE="${REGISTRY}/${IMAGE_NAME}"

echo "Building image..."
docker build -f "${DOCKERFILE_DIR}/Dockerfile" -t "${IMAGE}:${VERSION}" -t
```

```
"${IMAGE}" "${DOCKERFILE_DIR}"

echo "Pushing image tags..."
docker push "${IMAGE}:${VERSION}"
docker push "${IMAGE}"

echo
echo "Selesai. Image tersedia sebagai:"
echo "  ${IMAGE}:${VERSION}"
echo "  ${IMAGE}"
```

untuk membuatnya kita langsung aja pake command

nano ~/latihan-dockerfile/build-push.sh

nah abis itu kita tinggal paste aja skrip diatas dan setelah itu jalankan commannd

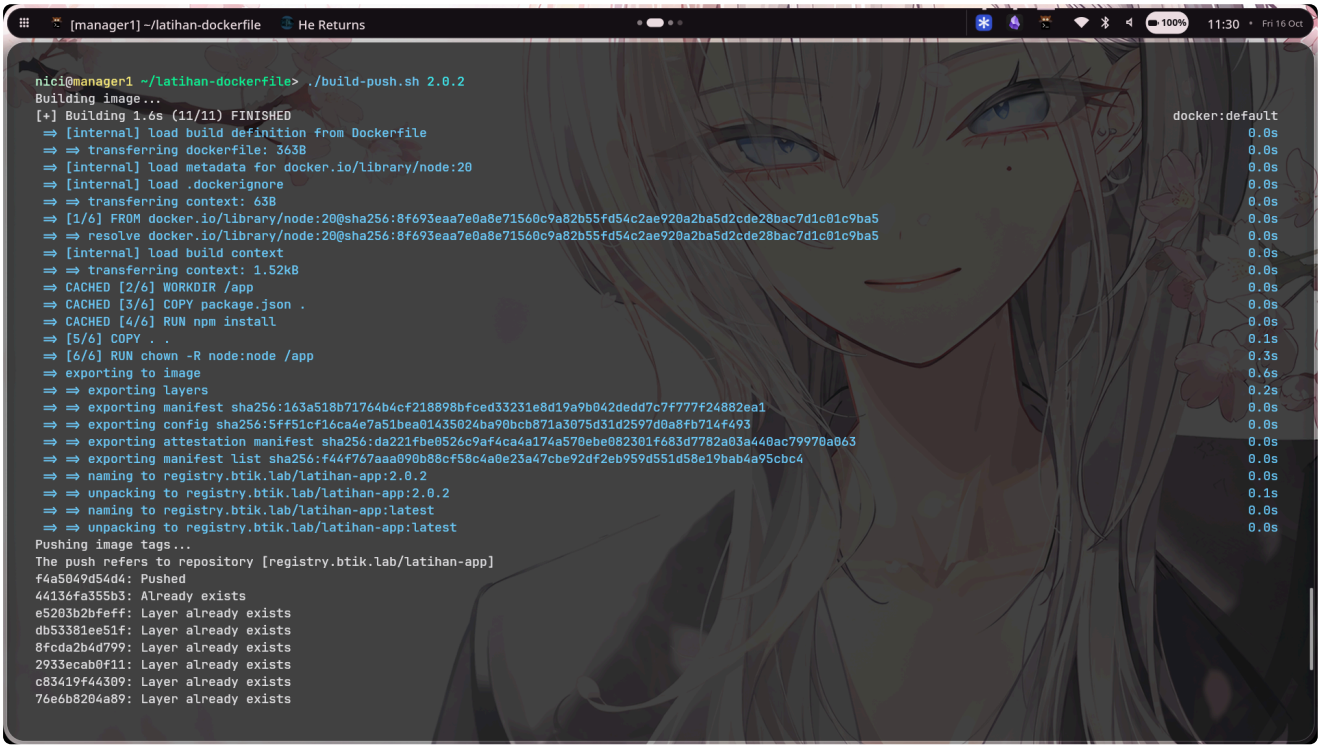
```
chmod +x ~/latihan-dockerfile/build-push.sh
```

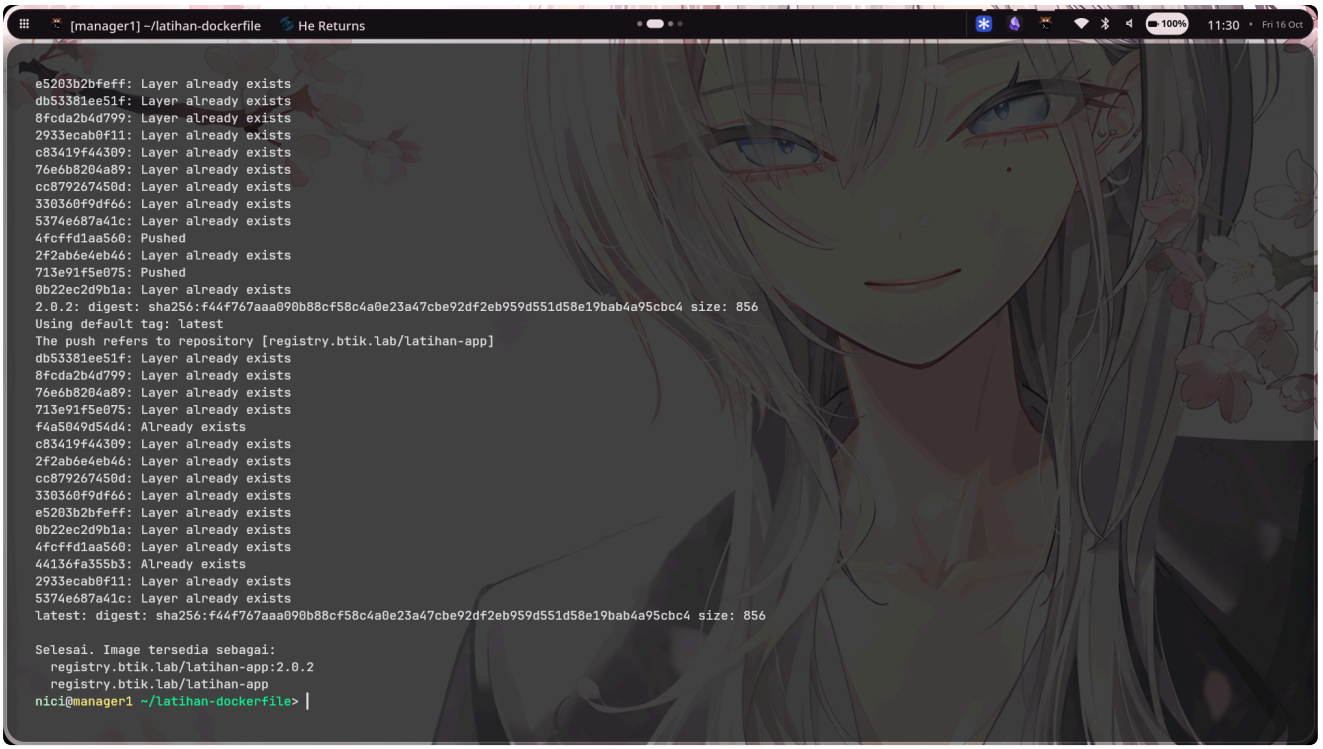
dan langsung aja jalankan dengan command

```
cd /latihan/dockerfile
```

```
./build-push.sh 2.0.1
```

yang mana kira kira hasilnya jadi





seperti itu dan kita juga bisa cek langsung pada registry nya apakah ada atau tidak hasil push dari skrip itu dengan command

```
curl --cacert ~/infra-ca.pem -u admin -sS \
https://registry.btik.lab/v2/latihan-app/tags/list
```

yang mana hasilnya kira kira kaya gini



nah disana muncul 2 tag baru, yaitu 2.0.1 dan 2.0.2 yang mana menandakan skripnya berhasil lalu topik selanjutnya adalah soal **kebijakan retensi lama**, intinya kita tidak bisa seenak jidat menghapus sebuah tag, maka dari itu kebijakan retensi ada untuk menentukan tag mana saja yang bisa dihapus dan tag mana saja yang tidak, nah untuk masalah itu kita akan gunakan pola yang umum, yaitu:

- Simpan N build terakhir per branch/environment (misal 10 build terakhir)
- Simpan semua tag semver (`1.0.0` , `1.0.1` , dst) selamanya, ini yang dipakai orang buat rollback manual, jangan diotak-atik otomatis
- Hapus tag git-SHA yang lebih tua dari X hari dan bukan yang sedang “live”/dipakai deployment manapun

lalu untuk skrip retensi sederhana kira kira seperti ini

```
#!/bin/bash
set -euo pipefail

REGISTRY="registry.btik.lab"
CA_CERT="$HOME/infra-ca.pem"
AUTH="admin"
IMAGE="latihan-app"
RETENTION_DAYS=30

echo "Mengambil daftar semua tag untuk ${IMAGE}..."

TAGS=$(curl --fail -sS \
    --cacert "$CA_CERT" \
    -u "$AUTH" \
    "https://${REGISTRY}/v2/${IMAGE}/tags/list" \
    | jq -r '.tags[]?')

for TAG in $TAGS; do
    # Skip tag semver (format x.y.z) dan latest
    if [[ "$TAG" =~ ^[0-9]+\.[0-9]+\.[0-9]+$ ]] \
        || [[ "$TAG" == "latest" ]]; then
        echo "Skip $TAG (tag penting, tidak dihapus otomatis)"
        continue
    fi

    echo "Kandidat hapus (tag non-semver): $TAG"
    # Penghapusan memerlukan pemeriksaan usia tag dan digest manifest.
done

echo "Selesai. Tidak ada tag yang dihapus."
```

lalu isi skrip itu? intinya adalah skrip simulasi kebijakan retensi yang mana dia akan mengidentifikasi tag yang berpotensi dihapus dari registry agar registry tidak selalu menyimpan tag tag lama yang udah ga dibutuhkan, lalu next ini ke **script gate Trivy, batasi akses dengan ufw, tanda tangan Cosign**

nah ini intinya adalah untuk yang script gate trivy itu tuh dimana image hanya boleh di push kalau udah lolos scan trivy tanpa ada vulnerability yang critical atau high yang mana langsung aja kita buat skripnya yang kira kira seperti ini

```
#!/bin/bash
set -uo pipefail

REGISTRY="registry.btik.lab"
IMAGE_NAME="latihan-app"
VERSION="${1:-}"

if [ -z "$VERSION" ]; then
```

```
    echo "Usage: $0 <version>"
    exit 1
fi

LOCAL_TAG="${IMAGE_NAME}:${VERSION}"
REMOTE_TAG="${REGISTRY}/${IMAGE_NAME}:${VERSION}"
LATEST_TAG="${REGISTRY}/${IMAGE_NAME}:latest"

mkdir -p "$HOME/trivy-tmp" "$HOME/.cache/trivy"

echo "=== Build image ==="
if ! docker build -t "$LOCAL_TAG" .; then
    echo "GAGAL: Build image."
    exit 1
fi

echo "=== Scan dengan Trivy ==="
if docker run --rm \
    -v /var/run/docker.sock:/var/run/docker.sock \
    -v "$HOME/trivy-tmp:/tmp/trivy-tmp" \
    -v "$HOME/.cache/trivy:/root/.cache/trivy" \
    -e TMPDIR=/tmp/trivy-tmp \
    aquasec/trivy:0.74.0 image \
    --skip-version-check \
    --scanners vuln \
    --exit-code 1 \
    --severity CRITICAL \
    "$LOCAL_TAG"
then
    echo "=== Scan lolos ==="
else
    SCAN_RESULT=$?
    echo "=== SCAN GAGAL ATAU DITOLAK ==="
    echo "Exit code Trivy: $SCAN_RESULT"
    echo "Push dibatalkan."
    exit 1
fi

echo "=== Tag image untuk registry ==="
docker tag "$LOCAL_TAG" "$REMOTE_TAG"
docker tag "$LOCAL_TAG" "$LATEST_TAG"

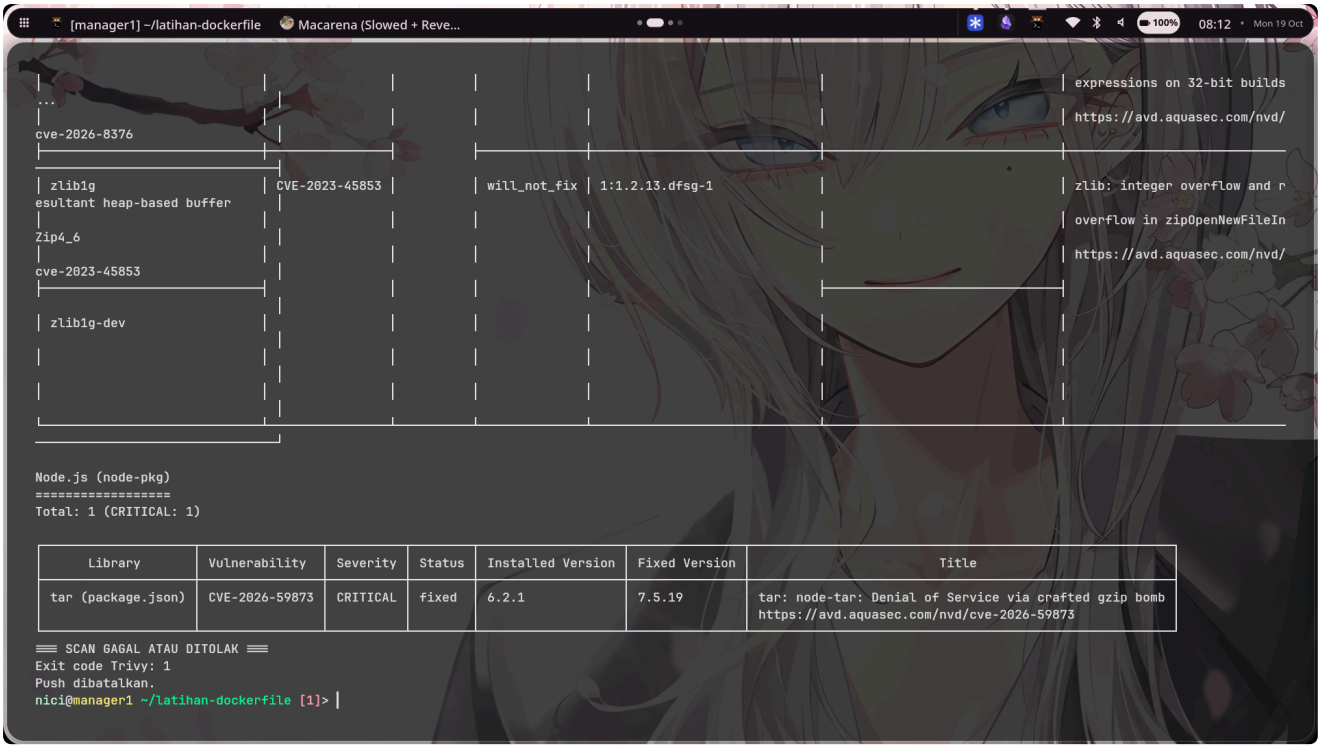
echo "=== Push image ==="
if ! docker push "$REMOTE_TAG"; then
    echo "GAGAL: Push tag versi."
    exit 1
fi

if ! docker push "$LATEST_TAG"; then
    echo "GAGAL: Push latest."
```

```
    exit 1
fi

echo "=== Selesai ==="
echo "Image tersedia sebagai:"
echo "  $REMOTE_TAG"
echo "  $LATEST_TAG"
```

dan saat aku jalankan kira kira hasilnya seperti ini

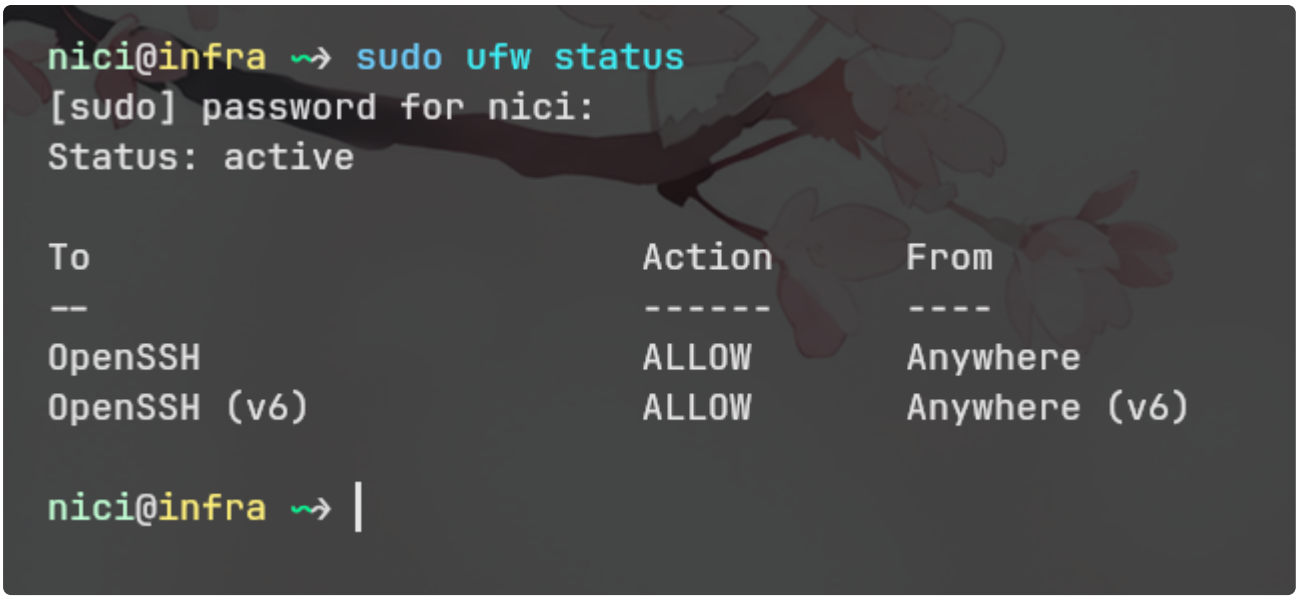


disitu terlihat bahwa image terdetekti ada vuln critical pada image tersebut dan image pun ditolak untuk di push

lalu untuk masalah membatasi akses dengan menggunakan ufw, kita akan ke vm infra karena disana terpasang registry maupun firewall utamanya jadi kita akan ke vm infra lalu langsung aja cek apakah ufw nya masih jalan dengan command

```
sudo ufw status
```

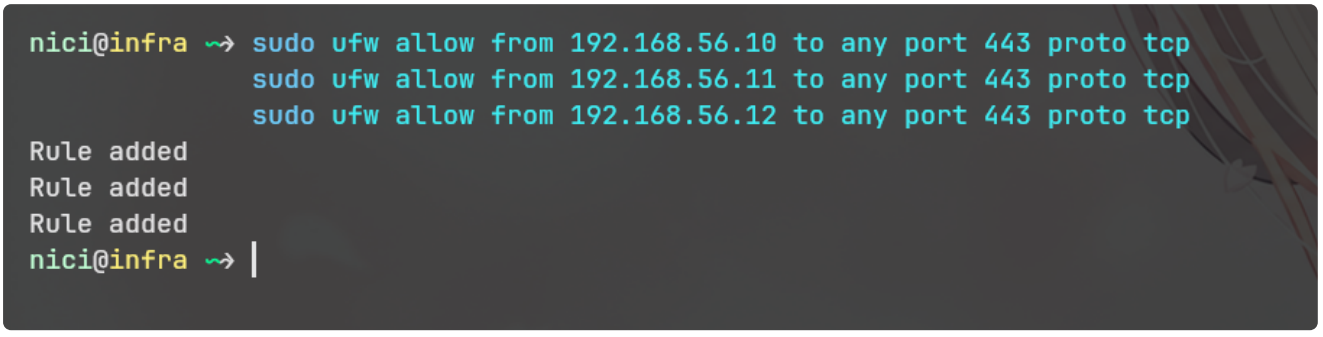
yang hasilnya adalah



lalu selanjutnya kita akan allow protokol 443 hanya dari beberapa ip dari vm yang dikenali, yaitu manager1, worker1, dan worker2 dengan command

```
sudo ufw allow from 192.168.56.10 to any port 443 proto tcp
sudo ufw allow from 192.168.56.11 to any port 443 proto tcp
sudo ufw allow from 192.168.56.12 to any port 443 proto tcp
```

lalu hasilnya adalah



harus rule added semua ya dan kenapa pakai "allow from" ? bukan hanya "allow" karena allow from itu jauh lebih ketat dan kalau allow saja itukan berarti hanya bisa konek ke port itu dan siapapun bisa konek, nah kalau allow from itu tuh lebih ketat, yaitu hanya ip yang di whitelist dan port yang bisa konek, selain itu tidak bisa konek sama sekali, nah kita juga akan verif apakah udah ter apply apa belum dengan command

```
sudo ufw status numbered
```

yang mana hasilnya kira kira seperti ini

```
nici@infra ⇝ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] OpenSSH                    ALLOW IN    Anywhere
[ 2] 443/tcp                    ALLOW IN    192.168.56.10
[ 3] 443/tcp                    ALLOW IN    192.168.56.11
[ 4] 443/tcp                    ALLOW IN    192.168.56.12
[ 5] OpenSSH (v6)               ALLOW IN    Anywhere (v6)

nici@infra ⇝
```

setelah itu kita juga akan coba tes ombak, karena ip host ku ga aku masukkan ke whitelist jadi aku akan coba konek ke infra melalui host, tapi ada suatu masalah yang mana secara teori, konfigurasi ufw diatas memang udah bener dan bisa nge block, cuma masalahnya ada pada registry yang berjalan di docker infra, kenapa ini jadi masalah? karena container registry itu berjalan melalui port mapping ini

```
0.0.0.0:443->443/tcp
[::]:443->443/tcp
```

yang mana artinya docker itu membuka port 443 ke semua host yang menggunakan ipv4 ataupun v6 dan docker juga ngatur NAT nya sendiri jadinya pada konfigurasi yang pake iptables, traffick port itu bisa langsung diarahkan sebelum melewati ufw nya jadi itu kenapa kalau pakai itu ufw saja, jaringan yang tidak di white list itu masih bisa tetap konek, lalu saat aku cek cek chain DOCKER-USER pake command iptables, awalnya hasilnya kosongan banget, kaya gini

```
nici@infra ⇝ sudo iptables -S DOCKER-USER
-N DOCKER-USER
-A DOCKER-USER -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
nici@infra ⇝
```

nah setelah itu kita akan menambahkan beberapa rule untuk mengizinkan koneksi dari vm manager1, worker1, dan worker2 dengan command

```
sudo iptables -A DOCKER-USER \
  -p tcp -s 192.168.56.10 \
  -m conntrack --ctorigdst 192.168.56.13 --ctorigdstport 443 \
  -j ACCEPT
```

lalu untuk worker1

```
sudo iptables -A DOCKER-USER \
  -p tcp -s 192.168.56.11 \
  -m conntrack --ctorigdst 192.168.56.13 --ctorigdstport 443 \
  -j ACCEPT
```

dan worker2

```
sudo iptables -A DOCKER-USER \
  -p tcp -s 192.168.56.12 \
  -m conntrack --ctorigdst 192.168.56.13 --ctorigdstport 443 \
  -j ACCEPT
```

dan kita juga harus menolak koneksi lain yang menuju ke registry, command yang digunakan adalah

```
sudo iptables -A DOCKER-USER \
  -p tcp \
  -m conntrack --ctorigdst 192.168.56.13 --ctorigdstport 443 \
  -j DROP
```

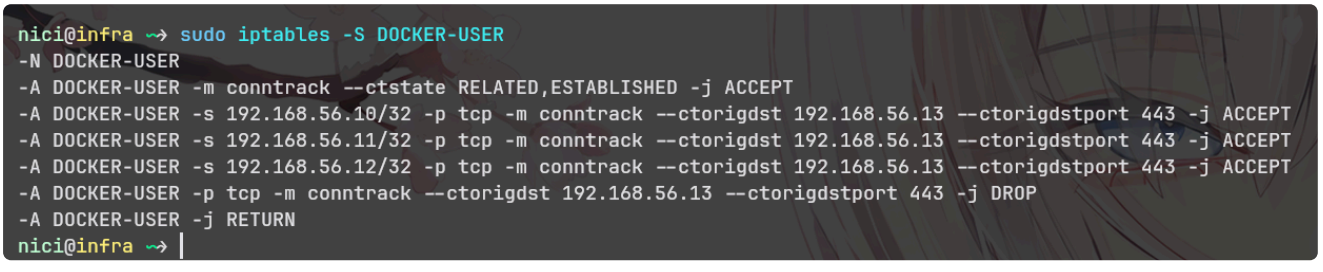
dan juga jangan lupa kita harus kembalikkan paket yg lain ke dalam chain dengan command

```
sudo iptables -A DOCKER-USER -j RETURN
```

nah kalau kita cek pakai command

```
sudo iptables -S DOCKER-USER
```

nanti hasilnya seperti ini



nah dan kita akan tek koneksi dari setiap vm yang terdaftar dan host ke registry dengan menggunakan command yang sama, yaitu

```
nc -vz -w 3 192.168.56.13 443
```

yang mana hasil dari manager1 yaitu

```
nici@manager1 ~/latihan-dockerfile> nc -vz -w 3 192.168.56.13 443
Connection to 192.168.56.13 443 port [tcp/https] succeeded!
nici@manager1 ~/latihan-dockerfile>
```

lalu worker1

```
nici@worker1 ~/latihan-dockerfile> nc -vz -w 3 192.168.56.13 443
Connection to 192.168.56.13 443 port [tcp/https] succeeded!
nici@worker1 ~/latihan-dockerfile>
```

lalu worker2

```
nici@worker2 ~> nc -vz -w 3 192.168.56.13 443
Connection to 192.168.56.13 443 port [tcp/https] succeeded!
nici@worker2 ~>
```

dan yang harusnya diblokir yaitu host,

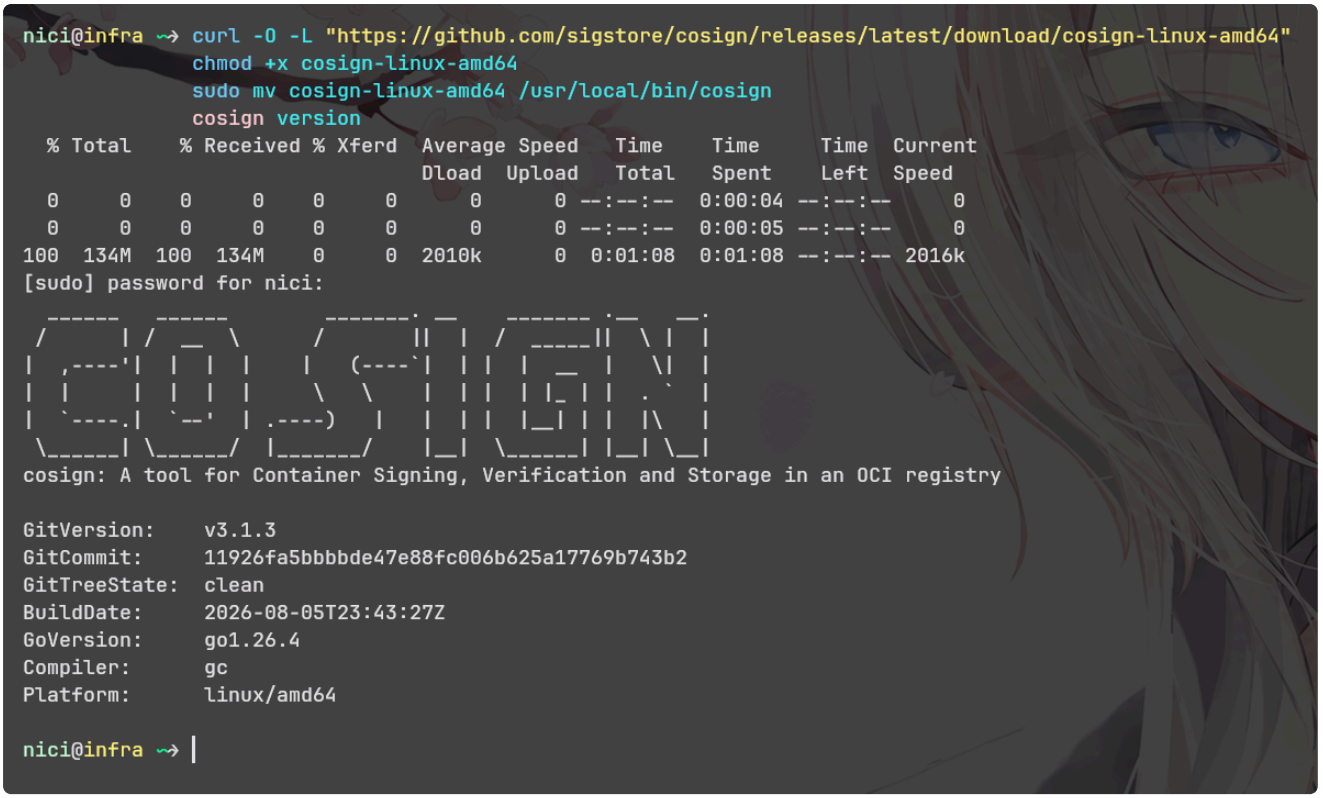
```
~ ❯ nc -vz -w 3 192.168.56.13 443                                    10:56
nc: connect to 192.168.56.13 port 443 (tcp) timed out: Operation now in progress
~ ❯
```

nah jadi untuk masalah pembatasan sudahh clear dan kita akan lanjut ke

tanda tangan image dengan menggunakan cosign, nah cosign sendiri untuk konsepnya sebenarnya cukup simpel, yaitu memberikan tanda tangan pada image kita, maksudnya gimana? well maksudnya adalah selama ini kita kan menggunakan TLS, yaitu koneksi https dan password saja yang mana orang yang ngepush pasti orang orang yang dibolehkan, tapi tidak ada yang bisa membuktikan kalau image yang di pull nanti itu benar benar di build oleh kita, bisa aja ada orang entah berantah yang dapat akses password nya dan nimpa build yang sebelumnya dengan build apalah itu jadi ya yang kena kita sendiri bisa bisa, nah untuk mengatasi hal tersebut kita gunakan cosign, ya simpelnya ini hanya untuk tanda tangan saja kalau image yang di build itu benar benar image yang di build oleh tim kita sendiri, bukan orang lain yang mana langsung aja kita akan install cosign dengan command

```
curl -O -L
"https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-
amd64"
chmod +x cosign-linux-amd64
sudo mv cosign-linux-amd64 /usr/local/bin/cosign
cosign version
```

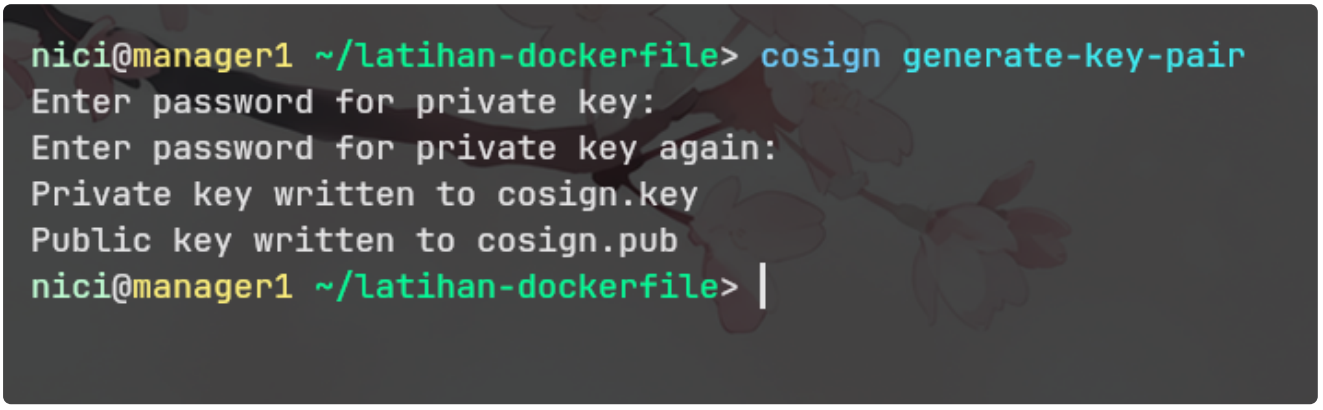
yang mana hasilnya akan seperti ini



lalu selanjutnya kita akan generate key pair dengan command

```
cosign generate-key-pair
```

yang mana nanti kita disuruh untuk masukan passphrase yang kira kira kalau udah sukses hasilnya seperti ini



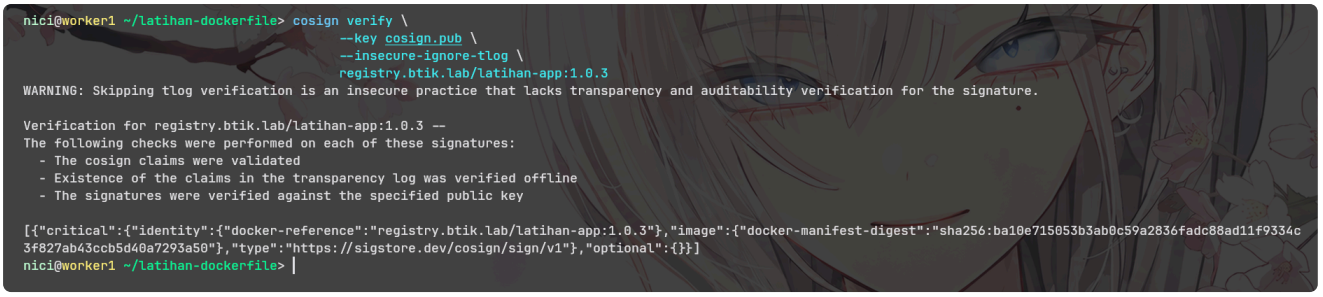
nah setelah itu kita akan langsung tanda tangani image yang udah di push dengan command

```
cosign sign --key cosign.key registry.btik.lab/latihan-app:1.0.3
```

lalu kita verivikasi dari vm lain dengan command

```
cosign verify --key cosign.pub registry.btik.lab/latihan-app:1.0.3
```

kira kira hasilnya seperti ini



nah kita sudah bisa verifikasi dan sebelum verif, pastikan pub key dari tiap node itu dibagi ke satu sama lain seperti hal nya ssh key ya, udah itu aja untuk task 8, makasihhh